

Chapter 18: Abstraction in Java

Personal Notes

Abstraction is the last of the four pillars of OOP. While encapsulation hides DATA, abstraction hides IMPLEMENTATION — the messy details of HOW something works — and exposes only what it does.

Every time you drive a car without knowing how the engine works, send a message without knowing how the internet routes it, or call a method without reading its code — you are using abstraction. This chapter shows how Java lets you build the same kind of clean interfaces in your own code.

1. What is Abstraction?

Abstraction is the practice of HIDING the complex internal details and EXPOSING only the essential parts. The user sees WHAT an object can do — not HOW it does it.

The Idea in One Line

Abstraction = "Show what, hide how."

Real-World Examples

- Driving a car — you use the steering, pedals, gear. You don't know how the engine pistons fire or how fuel injection works.
- Using an ATM — you press buttons to withdraw cash. You don't know how the database updates your balance or how the cash dispenser works.
- Sending a message on WhatsApp — you type and hit send. You don't see TCP packets, encryption, or server routing.
- Pressing a remote button — the TV turns on. You don't care about infrared signals or circuits.

In every case, you interact with a SIMPLE interface, while a COMPLEX system runs underneath. That is abstraction.

2. Encapsulation vs Abstraction

These two are often confused. They are related but different:

Aspect	Encapsulation	Abstraction
Hides	Data (the variables)	Implementation (the HOW)
Achieved by	private fields + getters/setters	abstract classes + interfaces
Goal	Protect data from misuse	Show only what's needed, hide complexity

Aspect	Encapsulation	Abstraction
Level	Inside a single class	Across a system / class hierarchy
Example	private balance + getBalance()	abstract draw() in Shape; Circle implements it

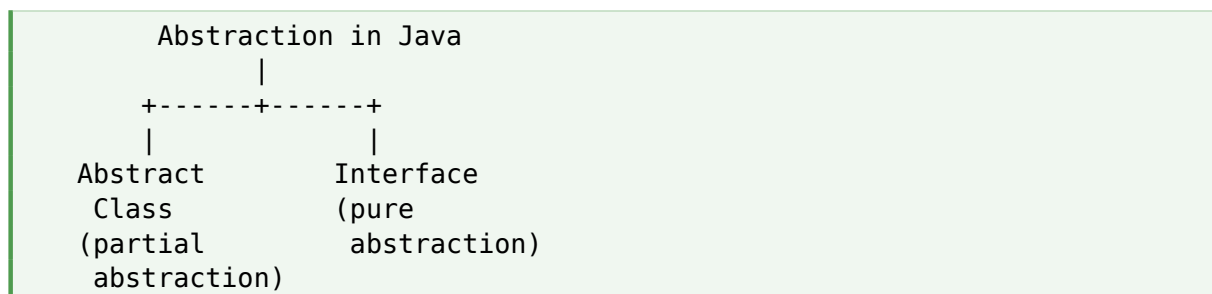
Quick way to remember: Encapsulation is the LOCKER — it hides the data. Abstraction is the BUTTON — it hides the wiring behind it.

3. Why Abstraction Matters

- Reduces Complexity — users see only what's relevant, not the internal mess
- Improves Readability — code reads at a higher, cleaner level
- Enhances Security — internal details stay hidden from the outside world
- Easier Maintenance — you can rewrite internals without breaking callers
- Promotes Reusability — abstract types can be implemented many different ways
- Enables Polymorphism — abstract methods are exactly what polymorphism needs to shine

4. How to Achieve Abstraction in Java

Java gives you two tools for abstraction:



- Abstract Classes — give PARTIAL abstraction. Can have both abstract methods (no body) and regular methods (with body).
- Interfaces — give PURE abstraction. Traditionally only method signatures, no implementation.

5. Abstract Classes

An abstract class is a class declared with the abstract keyword. It cannot be instantiated directly — you cannot create an object from it. It exists only to be extended by other classes.

5.1 Abstract Methods

An abstract method is a method with NO body — no implementation. It only has a signature. The actual code must be provided by any concrete child class.

```
abstract class Shape {  
    // abstract method – no body  
    abstract void draw();  
  
    // regular method – has a body, inherited as-is  
    void info() {  
        System.out.println("This is a shape.");  
    }  
}
```

5.2 Rules for Abstract Classes

- Declared with the abstract keyword
- Cannot be instantiated directly with new
- Can have BOTH abstract and concrete methods
- Can have constructors (called by child classes via super())
- Can have instance variables and static fields
- If a class has even ONE abstract method, the class MUST be declared abstract
- Any concrete child class MUST implement all inherited abstract methods

5.3 Full Example

```
abstract class Shape {  
    String color;  
  
    Shape(String color) {  
        this.color = color;  
    }  
  
    // abstract – children must implement this  
    abstract double area();  
  
    // concrete – shared by all children  
    void describe() {  
        System.out.println("This is a " + color + " shape.");  
    }  
}  
  
class Circle extends Shape {  
    double radius;  
  
    Circle(String color, double radius) {
```

```

        super(color);
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double length, width;

    Rectangle(String color, double length, double width) {
        super(color);
        this.length = length;
        this.width = width;
    }

    @Override
    double area() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle("Red", 5);
        s1.describe();
        System.out.println("Area: " + s1.area());

        Shape s2 = new Rectangle("Blue", 4, 6);
        s2.describe();
        System.out.println("Area: " + s2.area());

        // Shape s3 = new Shape("Green");    // ERROR – can't instantiate
    }
}

```

Output:

```

This is a Red shape.
Area: 78.53981633974483
This is a Blue shape.
Area: 24.0

```

What this shows: The Shape class defines what every shape MUST be able to do (area()) and what they all SHARE (describe()). Each subclass fills in the missing pieces. The user just calls area() — they don't care which shape it is.

6. Interfaces

An interface in Java is the purest form of abstraction. Traditionally, it could contain ONLY method signatures — no fields, no implementations. It's a pure contract: "Here's what you must be able to do."

6.1 Basic Syntax

```
interface Drawable {
    void draw();    // implicitly public and abstract
}

class Circle implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
}

class Square implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing a square");
    }
}

public class Main {
    public static void main(String[] args) {
        Drawable d1 = new Circle();
        Drawable d2 = new Square();

        d1.draw();    // Drawing a circle
        d2.draw();    // Drawing a square
    }
}
```

6.2 Rules for Interfaces

- Declared with the interface keyword (not class)
- All methods are PUBLIC and ABSTRACT by default (no need to write those keywords)
- All fields are public, static, and final by default — they are constants
- Classes use the implements keyword (not extends) to use an interface
- A class can implement MANY interfaces — this is how Java does multiple inheritance
- An interface can extend another interface using extends
- Cannot be instantiated directly

6.3 default and static Methods (Java 8+)

From Java 8, interfaces gained the ability to have method bodies through default and static methods. This was added to let APIs evolve without breaking existing implementations.

```
interface Vehicle {
    void start();    // abstract – must be implemented

    default void honk() {           // default – has a body
        System.out.println("Beep beep!");
    }

    static void info() {           // static – called via interface
name
        System.out.println("Vehicles transport people.");
    }
}

class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car engine started");
    }
    // honk() is inherited from the interface as-is
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.start();
        c.honk();           // Beep beep!
        Vehicle.info();     // call static directly on interface
    }
}
```

7. Multiple Inheritance Through Interfaces

Java does not allow a class to extend more than one class (to avoid the Diamond Problem). But a class CAN implement many interfaces — this is how Java achieves multiple-inheritance behavior safely.

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {
```

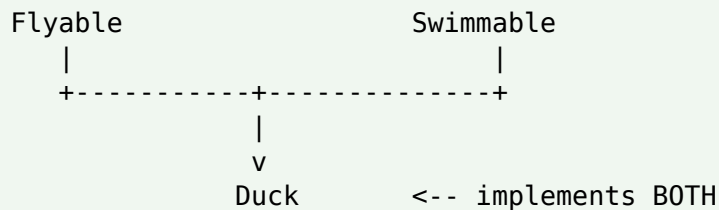
```

    @Override
    public void fly() {
        System.out.println("Duck is flying");
    }

    @Override
    public void swim() {
        System.out.println("Duck is swimming");
    }
}

public class Main {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();
        d.swim();
    }
}

```



Why this works: Interfaces traditionally have no implementation, so there's no conflict when a class implements two of them. Even with default methods, if there's an ambiguity, Java requires you to resolve it explicitly — no silent confusion.

8. Abstract Class vs Interface

Aspect	Abstract Class	Interface
Keyword	abstract class	interface
Extended by	extends	implements
Multiple parents	Only ONE abstract class can be extended	MANY interfaces can be implemented
Methods	Abstract AND concrete methods	Abstract methods (+ default/static since Java 8)
Fields	Any type — instance, static, final	Only public static final (constants)
Constructors	Yes	No
Access modifiers	Any — public, protected, private	Methods are public by default

Aspect	Abstract Class	Interface
Use when	Classes share some COMMON code	Classes only share a CONTRACT, not code

Rule of thumb: Use an abstract class when subclasses share BOTH some implementation and some required-but-different behavior. Use an interface when you only care about WHAT a class can do, not how. When in doubt, prefer interfaces — they're more flexible.

9. Real-World Examples

9.1 Payment System

Different payment methods all do the same thing (pay) but in different ways. Perfect fit for abstraction.

```

abstract class Payment {
    double amount;

    Payment(double amount) {
        this.amount = amount;
    }

    abstract void pay();    // each method does it differently

    void receipt() {        // shared logic
        System.out.println("Payment of " + amount + " processed.");
    }
}

class CreditCard extends Payment {
    CreditCard(double amount) { super(amount); }

    @Override
    void pay() {
        System.out.println("Paid " + amount + " via Credit Card");
    }
}

class UPI extends Payment {
    UPI(double amount) { super(amount); }

    @Override
    void pay() {
        System.out.println("Paid " + amount + " via UPI");
    }
}

```

```

class Cash extends Payment {
    Cash(double amount) { super(amount); }

    @Override
    void pay() {
        System.out.println("Paid " + amount + " in Cash");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment p1 = new CreditCard(1000);
        Payment p2 = new UPI(500);
        Payment p3 = new Cash(200);

        p1.pay();
        p2.pay();
        p3.pay();
        p3.receipt();
    }
}

```

9.2 The TV Remote (Mental Model)

What you see
(the public interface)

```

+-----+
|  ⏻    |
| CH▲ CH▼ |
| VOL▲ VOL▼ |
+-----+

```

You only press buttons.
The internal circuits stay hidden.
That is abstraction.

What is hidden
(the implementation)

Infrared signals
Microcontroller logic
Frequency tuning
Display drivers
Audio decoders

10. Common Mistakes

Mistake 1: Trying to instantiate an abstract class

```

Shape s = new Shape();    // ERROR
Shape s = new Circle();   // OK – concrete subclass

```

Mistake 2: Forgetting to implement all abstract methods

Any concrete child class MUST implement EVERY abstract method it inherits. Forgetting even one causes a compile error — unless you declare the child as abstract too.

Mistake 3: Trying to give an abstract method a body

```
abstract void draw() {  
    System.out.println("...");    // ERROR – abstract methods can't have  
    bodies  
}
```

Mistake 4: Using extends with interfaces

```
class Circle extends Drawable { ... }    // WRONG  
class Circle implements Drawable { ... } // correct
```

Use extends for classes (and interfaces extending interfaces), implements for classes implementing interfaces.

Mistake 5: Putting non-final fields in an interface

All fields in an interface are public static final by default — you cannot have regular instance fields. If you need instance state, use an abstract class.

Mistake 6: Confusing abstraction with encapsulation

Encapsulation hides DATA inside a class. Abstraction hides IMPLEMENTATION behind an interface. Both involve hiding, but at different levels.

11. Best Practices

- Program to interfaces, not implementations — variables of type Shape, not Circle
- Prefer interfaces when classes only need to share a contract, not code
- Use abstract classes when there's shared state or shared implementation
- Keep interfaces small and focused — many small interfaces beat one giant one
- Always use @Override when implementing or overriding methods
- Make abstract methods describe WHAT, not HOW — let subclasses decide how
- Don't over-abstract — abstraction has a cost (more files, more indirection). Only abstract when there are real variations to model.

12. Quick Reference

Term / Keyword	Meaning
Abstraction	Hiding implementation, exposing only essential behavior
abstract class	Class that cannot be instantiated; meant to be extended
abstract method	Method with no body; subclass must implement
interface	Pure contract — defines WHAT, not HOW

Term / Keyword	Meaning
extends	Used by a class to inherit from another class or interface
implements	Used by a class to commit to an interface's contract
default method	Interface method with a body (Java 8+)
static method	Interface method tied to the interface itself (Java 8+)
@Override	Annotation marking a method as overriding/implementing
Multiple inheritance	Allowed only through interfaces, not classes
Diamond Problem	The ambiguity Java's class rules are designed to prevent

13. Practice Problems

Try these to lock in both abstract classes and interfaces.

1. Create an abstract `Animal` class with abstract method `sound()` and concrete method `sleep()`. Build `Dog`, `Cat`, and `Cow` subclasses.
2. Design a `Vehicle` hierarchy: abstract `Vehicle` with `start()` and `stop()`. Implement `Car`, `Bike`, and `Truck`.
3. Create an interface `Playable` with `play()` and `pause()`. Implement it in `Music`, `Video`, and `Podcast` classes.
4. Build a payment system with abstract `Payment` class. Implement at least 3 payment methods (`CreditCard`, `UPI`, `NetBanking`).
5. Create an interface `Comparable` with `compareTo(Object other)`. Implement it in your own `Student` class.
6. Design two interfaces `Flyable` and `Swimmable`. Create a `Duck` class that implements both.
7. Build a `Shape` hierarchy with abstract `area()` and `perimeter()`. Implement `Circle`, `Square`, and `Triangle`.
8. Create an abstract `Employee` class with abstract `calculateSalary()`. Implement `Manager`, `Developer`, and `Intern` with different salary formulas.
9. Practice default methods: create an interface with one abstract and one default method, and implement it in two different classes.

14. Final Takeaway

Abstraction is what lets you build large software without drowning in details. By exposing only what matters and hiding the rest, you create code that other developers (including future-you) can actually use without reading every line.

Every Java framework you'll ever use — Spring, Hibernate, Android SDK, the Collections API — is built on abstract classes and interfaces. Once you internalize abstraction, those frameworks stop feeling like magic and start feeling like cleanly designed contracts.

Key Takeaway: Abstraction is "show what, hide how." Use abstract classes when subclasses share both contract and code. Use interfaces when they share only a contract. Program to abstractions, not concrete classes, and your code becomes flexible enough to grow without breaking.

15. What's Next?

With all four OOP pillars now in place (Encapsulation, Inheritance, Polymorphism, Abstraction), the next topics that build on them are:

- Generics — type-safe abstractions that work for any data type
- Collections framework — List, Set, Map (all built on interfaces)
- Exception handling — try, catch, finally, custom exceptions
- Lambda expressions and functional interfaces (Java 8+)
- Design Patterns — Strategy, Factory, Observer, Template Method
- SOLID principles — the design rules that build on abstraction